

CODA Features Deep Dive

Features Deep Dive

What's actually built, and exactly how it works — the technology choices and the logic behind each, not just a feature list. Written 2026-08-31. For the honest completion percentage and what's *not* built, see `docs/checkpoint_50.md` and `plan.md`'s "Known Gaps and Debt" section — this file is about the mechanics of what exists, not a status report.

1. Auth: JWT + rotating refresh tokens

Stack: `go-api`, `internal/auth`, Argon2id (`golang.org/x/crypto/argon2`), HS256 JWT.

Why not just a JWT with a long expiry? A JWT can't be revoked before it expires — once signed, it's valid until its `exp` claim says otherwise, no matter what the server does. "Logout" would be theater. So access tokens are short-lived (15 min) and carry the actual authorization (`role`, `org`), while a separate, **opaque random refresh token** (not a JWT — just random bytes) is the thing that can be revoked.

The rotation mechanism: the refresh token is never stored in Postgres in cleartext — only its SHA-256 hash. On every refresh, the old token is invalidated and a *new* one is issued, chained via `family_id / replaced_by_id` columns. If a client ever presents a refresh token that's already been replaced (meaning someone — an attacker with a leaked token — is using a stale one), that's a signal the whole chain is compromised, and the **entire family** is revoked, not just that one token. This is standard practice for revocable sessions, not a novel design, but it's why there are two separate token types instead of one.

Password hashing: Argon2id, not bcrypt — the memory-hard variant, deliberately chosen (bcrypt was the original scaffold's choice, switched before any real auth handlers existed).

2. RBAC + audit logging

Stack: `chi` middleware (`internal/auth.RequireRole`, `internal/audit.Middleware`).

Four roles: `admin`, `doctor`, `reviewer`, `auditor`. Enforcement happens at **two layers**, not one:

1. **Route-level:** `r.With(auth.RequireRole(auth.RoleDoctor)).Post(...)` — a role check before the handler even runs.
2. **Handler-level org-scoping:** `auth.RequireSameOrg(w, claims, resource.OrgID)` — even a valid `doctor` token from **Org A** gets a 403 (or a 404, for genuinely private resources) if it tries to touch **Org B**'s consultation. This is the layer that actually stops cross-tenant data leakage; the route-level role check alone wouldn't catch it.

Audit logging: a middleware wraps every mutating request and every clinical-data read, writing one row per request into `audit_log` — actor, action, resource, org, source IP, outcome (success/failure). The table has a Postgres trigger that rejects `UPDATE / DELETE` on it entirely, so it's genuinely append-only, not just append-only by convention.

3. The consultation/upload flow — presigned URLs, never proxied bytes

Stack: MinIO (S3-compatible object storage), `internal/storage` (Go MinIO SDK wrapper).

The core rule this follows: `go-api` must never stream audio bytes through itself. If it did, every upload would tie up a Go HTTP handler for however long the upload takes, and the server's own bandwidth becomes the bottleneck. Instead:

1. `POST /consultations/{id}/audio/presign` — the client says "I want to upload N bytes of type audio/wav with this SHA-256." `go-api` asks MinIO to **sign a PUT URL** that's valid for a short window and returns it — `go-api` never touches the actual audio.
2. The **browser** uploads directly to that signed URL. `go-api` is out of the loop entirely for the actual transfer.
3. `POST /consultations/{id}/audio/confirm` — the client reports back the object key + hash + duration. `go-api` calls MinIO's `StatObject` (a metadata-only call) to cross-check the **size** matches what was declared — the one integrity signal it can get without downloading and rehashing the whole file itself (which would defeat the entire point of not proxying bytes).

The two-endpoint MinIO problem (a real bug found and fixed this project): a presigned URL is signed against a specific hostname. `go-api` talks to MinIO over the Docker-network hostname (`minio:9000`), but a browser can't resolve that. The fix: `storage.Client` holds **two** MinIO SDK clients — one for `go-api`'s own calls (Docker hostname), one whose *only* job is signing URLs against a browser-reachable hostname (`MINIO_PUBLIC_ENDPOINT`, e.g. `localhost:9010`).

Same presigned-URL pattern is reused for downloads: `GET`

`/consultations/{id}/transcript` doesn't return transcript JSON itself — it returns a signed GET URL pointing straight at the MinIO object, and the frontend fetches it directly.

4. The pipeline orchestrator — the actual state machine

Stack: `go-orchestrator` (a separate binary from `go-api`), Postgres, Redis Streams.

This is the part doing the real coordination work. A consultation moves through a fixed sequence:

```
CREATED → CONSENT_RECORDED → UPLOADED → ASR_QUEUED → ASR_RUNNING → ASR_DONE  
      → REDACT_RUNNING → REDACTED → NLP_QUEUED → NLP_RUNNING → NLP_DONE  
      → AWAITING_REVIEW → UNDER_REVIEW → APPROVED → EXPORTED
```

with `FAILED` / `DEAD_LETTERED` / `CANCELLED` as terminal off-ramps from anywhere. **Redaction is structurally unskippable** — there is no code path from `ASR_DONE` to `NLP_QUEUED` that doesn't pass through `REDACT_RUNNING` first. This matters because redaction is meant to be the last checkpoint before transcript text reaches a third-party LLM (even though, as noted in §9, the redaction step itself doesn't yet do anything real).

Why Redis Streams and not a plain queue (SQS-style, or a Go channel)? Three properties a plain queue doesn't give you for free: - **Consumer groups** (`XREADGROUP`) — multiple workers can consume the same stream without duplicating work, and each message has an owner. - **A Pending Entries List (PEL)** — every delivered-but-unacknowledged message is tracked. If a worker crashes mid-processing, the message doesn't vanish; it sits in the PEL until `XAUTOCLAIM` reclaims it after a visibility timeout. - **At-least-once delivery with idempotent processing** — every dispatched envelope carries an idempotency key = `sha256(consultation_id || stage || run_config_id || input_sha256)`, enforced by a Postgres `UNIQUE` constraint on `job_stages.idempotency_key`. A redelivered message (crash recovery, a network blip) hits that constraint and updates the *same* row instead of re-running an expensive stage twice — this is what makes “a worker died mid-ASR-transcription” cost a retry, not a repeated 9-minute transcription paid for twice.

Retry policy is data, not code — `queue.PolicyFor(stage, options)` returns a `SoftDeadline`, `VisibilityTimeout`, and `MaxAttempts` per stage, derived from the job's own persisted `RunConfig` (never an environment variable), so a result's operational parameters are reconstructible from a database dump alongside everything else. Concretely: local-CPU ASR gets a 20-minute deadline (40-minute reclaim window), redaction/export get 60 seconds, single-pass NLP gets 10 minutes, and a future GoT arm gets 45 minutes (since free-tier LLM throttling, not compute, would dominate its wall-clock).

Backoff: not plain exponential — **full jitter:** `delay = random(0, min(120s, 2s × 2^(attempt-1)))`. The reasoning is specific to this system's actual failure mode: retryable failures are almost always provider-side (a Groq 5xx, model overload), which means every in-flight job in a batch tends

to fail at the *same instant*. Equal backoffs would just resynchronize them into another simultaneous retry wave; sampling uniformly from a window spreads them out.

RunConfig interning: each distinct ablation-arm configuration (model names, temperature, GoT knobs, knowledge-augmentation toggle) is content-hash-deduplicated — `sha256` over a *canonicalized* JSON serialization (deliberately re-marshaled through `encoding/json`, not raw `protojson` output, after a real bug where the latter wasn't guaranteed byte-stable across calls). Two jobs run under the identical config share one `run_configs` row; this is also what lets a resubmitted job reuse a *successfully completed* predecessor's stage result instead of recomputing it.

5. ASR pipeline (`asr-service`) — the real audio-to-transcript path

Stack: Python 3.11, `faster-whisper` (CTranslate2 runtime), `pyannote.audio` 3.1, Groq.

This runs as one of the two Python “workers” — a standalone process consuming a Redis Stream (`stage.asr`), not something `go-api` / `go-orchestrator` calls directly.

Step 1 — preprocessing (`audio.py`): decode via `pydub` / `ffmpeg`, reject anything that's the wrong format, corrupt, silent, or outside a configured duration range (fails loudly with a typed error rather than silently producing garbage), downmix to mono, resample to 16kHz, apply RMS-gain loudness normalization to a target dBFS.

Step 2 — transcription (`transcribe.py`): `faster-whisper`, model size `medium`, `int8` quantization, CPU-only (measured real-time factor $\approx 0.5\times$ — it transcribes faster than the audio plays, even on CPU). Long audio is **chunked** at a fixed 300-second window with 5-second overlap (only when audio exceeds $1.2\times$ that length), each chunk transcribed independently with `condition_on_previous_text=False` specifically to avoid a hallucinated continuation bleeding across the exact region the merge logic has to get right — words from the overlapping region are deduplicated by splitting at its temporal midpoint. This chunking exists purely for bounded per-call memory/time on a CPU laptop and to make heartbeat progress meaningful on long audio; `faster-whisper` doesn't correctness-wise need it.

Step 3 — diarization (`diarize.py`): `pyannote.audio`'s pretrained `speaker-diarization-3.1` pipeline runs over the same in-memory waveform, producing time-stamped speaker segments (Speaker A / Speaker B) — no clinical knowledge involved here, just “who was talking when.”

Step 4 — alignment (`align.py`), the WhisperX-style step that stitches steps 2 and 3 together: for each transcribed word, compute its temporal overlap against every diarization segment — `overlap`

`= min(word.end, seg.end) - max(word.start, seg.start)` — and assign the word to whichever segment overlaps it most. If no segment overlaps at all (a genuine diarization gap, or a word right at a VAD-trimmed silence boundary), fall back to whichever segment's edge is closest to the word's midpoint. Consecutive same-speaker words are grouped into turns.

Step 5 — role classification (`roles.py`): one Groq call classifies which diarized cluster is *Doctor* vs *Patient* from the cluster's own content — a few-shot prompt, not a rule, since “who talks more” or “who asks more questions” isn't reliable enough on its own. Both clusters are classified in a **single** call (not one call per cluster) specifically so the model can be told “don't assign the same role to both” and so `roles.py` can catch and downgrade to `uncertain` if it does anyway (per-cluster independent classification structurally can't detect that failure mode; one joint call can).

Everything above runs inside a custom asyncio worker harness (`coda_worker_sdk`, shared with `nlp-service`): an `XREADGROUP` consumer loop, a periodic `XAUTOCLAIM` reclaim sweep for crash recovery, a `Heartbeater` publishing progress every 15 seconds (this is what the frontend's live job-status page is actually reading, via Postgres — the orchestrator consumes the heartbeat stream and writes it to a row `go-api` can poll, since `go-api` itself is never allowed to touch the Redis streams directly).

6. NLP pipeline (`nlp-service`) — extraction and summary

Stack: Python 3.11, Groq (`qwen/qwen3.8-27b` for extraction/summary).

Redaction stage — currently an echo, not real. `_handle_redact` does exactly `turn.text_redacted = turn.text`, unchanged, for every turn. No pattern matching, no NER, no placeholder substitution. This is the most important caveat in this entire document: extraction and summary correctly read *only* `turn.text_redacted` (never `turn.text`), which is the right code pattern — but since that field is currently just an unredacted copy, real PII genuinely reaches Groq today. See `docs/checkpoint_50.md` §5.1 for the full disclosure; it's flagged prominently, not hidden.

Single-pass extraction (`extraction.py`) — this is the *baseline arm*, the experimental control the eventual GoT reasoning engine has to beat, not the reasoning engine itself: 1. One prompted Groq call against a **versioned prompt file** (`prompts/en/v1/extraction_system.md` / `extraction_user.md` — never an inline string literal in code, specifically so prompts version independently of code and so a v2 language variant is a new file, not a rewrite). 2. The model's JSON response is validated against a strict schema (`schema.py`). On failure, a **bounded repair loop** re-prompts with the validation error appended (`repair_addendum.md`), up to 2 attempts by default, before giving up with a typed fatal error — this validity rate is itself recorded as a metric

(`schema_valid`, `repair_attempts`), not silently patched over. 3. Every LLM call — including repair attempts — is routed through a **Postgres-backed cache** keyed on (`model`, `sha256(system_prompt + user_prompt)`). A retry with the identical prompt costs zero tokens; a repair retry's *different* prompt (the error text changed) correctly misses the cache and re-calls.

Summary (`summary.py`): a separate Groq call, sanity-length-validated only (no schema/repair loop — it's free text, not structured JSON).

Persistence: writes real rows into `extractions` (one row per field, medications/allergies split per a DB check constraint), `summaries`, and `clinical_notes` (`status='draft'` always — nothing has ever been approved by anything in this system yet).

7. Eval harness (`python/eval`) — how the measured numbers are actually computed

Stack: `jiwer` (WER/CER), `pyannote.metrics` (DER), `rapidfuzz`, `sentence-transformers` (MiniLM), `rouge-score`, `bert-score`, Groq (for the hallucination judge).

Datasets: PriMock57 (real recorded consultations + real clinician notes + speaker-labeled transcripts, an 8-of-57 subset actually fetched via `git lfs`), MTS-Dialog (1,701 dialogue/note pairs), ACI-Bench (207 encounters) — all real public corpora, no synthetic data.

WER/CER/DER aggregation — a specific, deliberate choice: computed as *total edits over total reference units* across the whole corpus (total word errors ÷ total reference words), **never a mean of per-item ratios**. The reason: a mean of ratios over-weights short items — a 10-word item with 1 error (WER 0.1) would count exactly as much as a 2,000-word item with 100 errors (WER 0.05), even though the second item is ten times the evidence. Both metric modules have a unit test that specifically asserts the aggregate differs from the naive mean, to catch a future refactor that accidentally reintroduces the naive version.

Field-level scoring (`field_match.py` / `field_scoring.py`) — three different matching rules, chosen per field, not one rule for everything: - **exact** (case/whitespace-normalized string equality) — reserved for `allergies` specifically, because a fuzzy false-match between two short, safety-critical strings (“penicillin” vs “amoxicillin”) is a dangerous failure mode a clinical eval must never paper over with leniency. - **normalized** (punctuation-normalized + `rapidfuzz` fuzzy-ratio threshold) — for short, fairly canonical phrases (`past_medical_history`, `medications`, `investigations_advised`) where phrasing varies but the underlying vocabulary doesn't need semantic understanding to reconcile. - **semantic** (cosine similarity of local `all-MiniLM-L6-v2` sentence embeddings, no API cost) — for free-text fields (`chief_complaint`, `hopi`,

`examination_findings`, `treatment_plan`, `provisional_diagnosis`) where clinical synonymy (“URTI” vs “upper respiratory tract infection”) or narrative paraphrase would make a string-level rule systematically undercount correct extractions.

A **wrong-but-present** value is deliberately scored as one false positive *and* one false negative, not a no-op — otherwise a chatty extractor that guesses something for every field would score better than an honest one that abstains when uncertain.

Hallucination judging (`hallucination.py`): an LLM-as-judge call (a separate Groq model, `openai/gpt-oss-20b`, distinct from the model under test — using the same model to grade its own answer would be circular). Sends the judge **only the transcript turns at least one field claim actually cites** — not the full transcript — both because citation-faithfulness is inherently a claim-against-its-own-cited-turns question, and because a real PriMock57 transcript alone (80-140 turns) was found live to exceed Groq’s free-tier per-minute token cap before any claims or rubric text were even added.

Real measured numbers from an actual run (not synthetic, not projected): ASR — WER 0.2226, CER 0.1666 on real PriMock57 audio through the real `asr-service` pipeline. Baseline extraction — field F1 0.6612 (ranging from 1.0 on `treatment_plan/examination_findings` down to 0.29–0.40 on `investigations_advised/medications` — a genuine, uneven signal, not a suspiciously uniform one), ROUGE-L 0.3440, BERTScore 0.8531, hallucination rate 0.2088.

8. Frontend — React 18 + Vite 5 + TypeScript

Stack: TanStack Query, `openapi-fetch` + `openapi-typescript`, `react-router-dom` v7.

Typed API client: `openapi-typescript` generates TypeScript types directly from the backend’s OpenAPI spec (`openapi/coda-v1.yaml`); `openapi-fetch` wraps `fetch` with those types, so a change to the API’s shape becomes a compile error in the frontend, not a runtime surprise.

Auth: JWT stored in `localStorage`. A custom `fetch` wrapper (not middleware, since middleware can’t cleanly retry a request) injects the `Authorization` header on every call except the auth endpoints themselves, and on a 401 triggers a token refresh (deduplicated — concurrent 401s don’t trigger multiple simultaneous refresh calls) before retrying the original request once.

Upload progress: deliberately `XMLHttpRequest`, not `fetch` — `fetch` has no upload-progress event at all; `XMLHttpRequest.upload.onprogress` is the only browser API that exposes it. SHA-256 of the file is computed client-side via the Web Crypto API before the upload starts, matching what the presigned upload flow (§3) needs.

Live job status — hand-rolled SSE, not the native `EventSource` : the native browser `EventSource` API cannot send custom headers, and this endpoint requires a Bearer token. Instead, a plain `fetch` call reads the response body as a stream, manually parsing `event: / data:` frames split on blank lines, ignoring `: -`-prefixed comment lines (the server's 15-second heartbeat). It reconnects with backoff on a dropped connection, and — after a real bug found this month — **does not treat a cleanly closed stream as “the job is done.”** A clean close only means “stop” once the caller has actually observed a terminal job state and calls `close()` itself; any other clean close (a timeout, a proxy, a load balancer) is now treated exactly like a dropped connection and reconnected.

Scope: deliberately read-only. There is no edit, approve, or export UI — a consultation can be uploaded, watched through processing, and its result viewed, and that's the whole loop today.

9. What doesn't exist yet, in one paragraph

The GoT-lite reasoning engine (Phase 6) — thought-graph construction, multi-candidate generation, rubric scoring, iterative refinement — is the actual research contribution of this project and none of it has been written. Everything in this document is the infrastructure built to make that phase measurable against a real, frozen, honestly-measured baseline once it exists — not a substitute for it. Knowledge augmentation (MeSH/ICD-10 entity linking, Phase 7) is likewise unbuilt. Real PII redaction (§6) is the highest-priority gap among what's already supposed to exist. See `plan.md`'s “Known Gaps and Debt” section for the full, priority-ordered list.